

Kinodynamic RRT Path Planning for Planar Hover-craft Robot

ECE 422 Final Project
Zhitao Yong

December 2024

1 Introduction

Path planning in robotics is a fundamental problem that enables autonomous systems to navigate from a start configuration to a goal configuration while avoiding obstacles. Traditional geometric path planning algorithms, such as the Rapidly-exploring Random Tree (RRT), operate in configuration space and assume that any path can be executed instantaneously. However, real-world robots are subject to physical constraints including dynamics, actuation limits, and inertia. These constraints make it impossible to execute arbitrary geometric paths, especially for systems with significant momentum or acceleration limits.

The problem becomes particularly important for applications involving:

- **Autonomous vehicles:** Cars, drones, and hovercrafts must plan trajectories that respect their dynamics, including acceleration limits, turning radius constraints, and momentum conservation.
- **Manipulator robots:** Industrial robots must plan motions that account for joint velocity and acceleration limits to ensure smooth, executable trajectories.
- **Spacecraft navigation:** Orbital mechanics and thruster limitations require kinodynamic planning to ensure feasible trajectories.
- **High-speed robotics:** Systems operating at high speeds cannot make instantaneous direction changes and must plan ahead to account for their momentum.

Kinodynamic RRT extends the classical RRT algorithm to **operate in the state space** (position and velocity) **rather than just configuration space**. This extension is crucial because it ensures that the planned paths are not only collision-free but also dynamically feasible, meaning they can be executed by a real robot given its physical constraints.

This project implements a Kinodynamic RRT planner for a planar hover-craft robot that can accelerate and decelerate in the x , y , and θ (orientation) dimensions. The implementation uses PyBullet for physics simulation and collision detection, ensuring that planned trajectories are both dynamically feasible and collision-free in a realistic environment.

2 Implementation

2.1 System Model

The hover-craft robot is modeled as a planar system with 6-dimensional state space:

$$\mathbf{x} = [x, y, \theta, v_x, v_y, v_\theta]^T \quad (1)$$

where (x, y) is the position, θ is the orientation, and (v_x, v_y, v_θ) are the corresponding velocities.

The system dynamics follow a double integrator model with acceleration control:

$$\dot{x} = v_x \quad (2)$$

$$\dot{y} = v_y \quad (3)$$

$$\dot{\theta} = v_\theta \quad (4)$$

$$\dot{v}_x = a_x \quad (5)$$

$$\dot{v}_y = a_y \quad (6)$$

$$\dot{v}_\theta = a_\theta \quad (7)$$

where $\mathbf{u} = [a_x, a_y, a_\theta]^T$ is the control input (acceleration in each dimension).

The discrete-time propagation with time step Δt is:

$$v_x(t + \Delta t) = v_x(t) + a_x \Delta t \quad (8)$$

$$v_y(t + \Delta t) = v_y(t) + a_y \Delta t \quad (9)$$

$$v_\theta(t + \Delta t) = v_\theta(t) + a_\theta \Delta t \quad (10)$$

$$x(t + \Delta t) = x(t) + v_x(t + \Delta t) \Delta t \quad (11)$$

$$y(t + \Delta t) = y(t) + v_y(t + \Delta t) \Delta t \quad (12)$$

$$\theta(t + \Delta t) = \theta(t) + v_\theta(t + \Delta t) \Delta t \quad (13)$$

Velocity limits are enforced to prevent unrealistic speeds:

$$|v_x|, |v_y| \leq v_{\max} = 3.0 \text{ m/s} \quad (14)$$

$$|v_\theta| \leq \omega_{\max} = 2.0 \text{ rad/s} \quad (15)$$

2.2 Kinodynamic RRT Algorithm

The Kinodynamic RRT algorithm extends the classical RRT by operating in the full state space and using motion primitives to generate dynamically feasible trajectories. The main algorithmic flow is illustrated in Figure 1 and described in Algorithm 2.

Kinodynamic RRT Algorithm Flow:

1. Initialize tree with start state
2. Sample random state in workspace
3. Find nearest node in tree (considering position and velocity)
4. For each motion primitive:
 - Propagate trajectory from nearest node
 - Check collision and bounds
 - Evaluate distance to target
5. Select best valid trajectory
6. Add new node to tree
7. Check if goal reached
8. Repeat until goal found or max iterations

Figure 1: Algorithm flow diagram for Kinodynamic RRT

```

FUNCTION RRT_KINODYNAMIC(x_start, x_goal, dt, goal_eps, max_iters):
  1. Initialize tree T with root node at x_start
  2. FOR k = 1 TO max_iters:
    a. Sample random state x_rand
    b. Find nearest node x_near in tree
    c. Try all motion primitives from x_near
    d. Select best valid trajectory
    e. Add new node to tree
    f. IF reached goal: RETURN path
  3. RETURN FAILURE

```

Figure 2: Kinodynamic RRT Algorithm Structure

2.3 Motion Primitives

Unlike geometric RRT which can directly steer toward a target, kinodynamic RRT must generate feasible trajectories using a discrete set of motion primitives. Each primitive is a constant acceleration control input $\mathbf{u} = [a_x, a_y, a_\theta]^T$ applied for a fixed duration.

The motion primitives are generated uniformly in the control space:

$$a_x, a_y \in \{-a_{\max}, \dots, 0, \dots, a_{\max}\} \quad (16)$$

where $a_{\max} = 2.0 \text{ m/s}^2$ is the maximum acceleration. The number of primitives per dimension is chosen to achieve the desired total number of primitives (4, 8, 16, or 32 in our experiments).

Design Decision: We chose a uniform grid distribution of primitives rather than random sampling because it provides better coverage of the control space and ensures that all directions are equally explored. This is particularly important for systems with symmetric dynamics like our hover-craft.

2.4 State Distance Metric

The distance metric for finding the nearest node must consider both position and velocity:

$$d(\mathbf{x}_1, \mathbf{x}_2) = w_p \|\mathbf{q}_1 - \mathbf{q}_2\| + w_v \|\mathbf{v}_1 - \mathbf{v}_2\| \quad (17)$$

where $\mathbf{q} = [x, y, \theta]^T$ is the configuration, $\mathbf{v} = [v_x, v_y, v_\theta]^T$ is the velocity, and $w_p = 1.0$, $w_v = 0.3$ are weights.

Design Decision: Position is weighted more heavily than velocity because reaching the goal position is the primary objective, while velocity matching is a secondary constraint. The angular difference in θ is computed using the shortest path on the circle.

2.5 Trajectory Propagation

For each motion primitive, the system state is propagated forward using the dynamics equations for $N = 10$ time steps of duration $\Delta t = 0.1 \text{ s}$, resulting in a 1-second trajectory segment. This creates a trajectory:

$$\tau = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N] \quad (18)$$

where $\mathbf{x}_0$ is the state at the nearest node and $\mathbf{x}_N$ is the final state.

Design Decision: We use **a fixed number of steps** rather than adaptive step sizing because it simplifies implementation and ensures consistent trajectory lengths. The 1-second duration provides a good balance between exploration distance and computational cost.

2.6 Collision Detection

Each trajectory is validated for collisions using PyBullet’s physics engine. The robot is represented as a box with dimensions $0.3 \times 0.3 \times 0.3$ m, and obstacles are cylindrical objects. For each state in the trajectory, the robot’s collision geometry is checked against all obstacles.

Design Decision: Using `PyBullet for collision detection` rather than simple geometric checks ensures accuracy and allows for future extensions to more complex robot shapes. The headless mode (DIRECT connection) is used during planning to maximize performance, while GUI mode can be enabled for visualization.

2.7 Goal Checking

A state is considered to have reached the goal if:

$$\|\mathbf{q} - \mathbf{q}_{\text{goal}}\| < \epsilon_p = 0.5 \text{ m} \quad (19)$$

$$\|\mathbf{v}\| < \epsilon_v = 0.5 \text{ m/s} \quad (20)$$

where $\mathbf{q}_{\text{goal}}$ is the goal position and the goal velocity is zero.

Design Decision: The goal region is defined as a circle in position space with a velocity tolerance. This allows the robot to reach the goal even if it cannot stop exactly at the target point, which is more realistic for systems with momentum.

2.8 Path Extraction

Once the goal is reached, the path is extracted by backtracing from the goal node to the root, collecting all trajectories along the way. The complete path consists of the concatenation of all trajectory segments:

$$\mathcal{P} = [\tau_0, \tau_1, \dots, \tau_K] \quad (21)$$

where each τ_i is a trajectory segment from one node to its child.

3 Results

3.1 Experimental Setup

The experimental environment consists of a 10×10 m workspace with 7 cylindrical obstacles arranged to create a challenging but solvable navigation problem. The obstacles are positioned at:

- (3, 2) with radius 0.6 m
- (5, 3) with radius 0.5 m
- (2, 5) with radius 0.6 m
- (6, 5) with radius 0.5 m
- (4, 7) with radius 0.5 m
- (7, 7) with radius 0.4 m
- (8, 5) with radius 0.5 m

The robot starts at position (1, 1) with zero velocity and must reach a goal region centered at (9, 9) with radius 0.8 m, also with zero velocity.

The algorithm parameters are:

- Time step: $\Delta t = 0.1$ s
- Maximum acceleration: $a_{\max} = 2.0$ m/s²
- Maximum velocity: $v_{\max} = 3.0$ m/s
- Maximum angular velocity: $\omega_{\max} = 2.0$ rad/s
- Position tolerance: $\epsilon_p = 0.5$ m
- Velocity tolerance: $\epsilon_v = 0.5$ m/s
- Maximum iterations: 8000

3.2 Main Demonstration Results

With 8 motion primitives, the algorithm successfully found a path in 0.28 seconds after exploring 194 nodes. The resulting path has a length of 16.97 m, which represents a reasonable trajectory that navigates around obstacles while respecting the robot’s dynamics.

The search tree visualization (available in `search.tree.png`) shows that the algorithm efficiently explores the free space, with the tree growing from the start position and eventually reaching the goal region. The final path (shown in green) demonstrates smooth navigation around obstacles. The trajectory execution animation (available in `trajectory.gif`) demonstrates the smooth, dynamically feasible motion of the robot along the planned path.

3.3 Motion Primitives Performance Evaluation

To understand the impact of the number of motion primitives on algorithm performance, we conducted experiments with 4, 8, 16, and 32 primitives. The results are summarized in Table 1.

Primitives	Success	Time (s)	Nodes	Path Length (m)
4	Yes	0.60	451	16.97
8	No	12.92	2946	N/A
16	Yes	8.65	2149	18.45
32	Yes	3.27	812	17.89

Table 1: Performance comparison for different numbers of motion primitives

Analysis of Results:

1. **4 Primitives:** This configuration achieved the fastest success time (0.60 s) but required exploring 451 nodes. The limited number of primitives constrains the robot’s ability to explore different directions, but the algorithm still succeeds because the environment is not overly constrained. The path length of 16.97 m is reasonable, though it may not be optimal.
2. **8 Primitives:** In this particular run, the algorithm failed to find a path within the iteration limit, exploring 2946 nodes in 12.92 seconds. This failure is likely due to the stochastic nature of RRT and the specific random samples generated. With more primitives, the algorithm has more options at each step, but this also increases computational cost per iteration. The failure suggests that 8 primitives may not provide sufficient coverage for this particular problem instance.
3. **16 Primitives:** The algorithm succeeded but required 8.65 seconds and explored 2149 nodes. The path length of 18.45 m is slightly longer than other successful runs, suggesting

that the algorithm may have taken a more circuitous route. The increased number of primitives provides better exploration but also increases the computational cost of evaluating each extension.

4. **32 Primitives:** This configuration achieved the best balance, succeeding in 3.27 seconds with only 812 nodes explored. The path length of 17.89 m is competitive with the 4-primitive case. The larger number of primitives allows the algorithm to find better trajectories more efficiently, reducing the total number of nodes needed to reach the goal.

Key Conclusions:

- **Too few primitives (4):** While fast, the limited control options may lead to suboptimal paths and reduced success rate in more constrained environments. The algorithm compensates by exploring more nodes.
- **Moderate primitives (8-16):** These configurations show inconsistent performance. The 8-primitive case failed in this run, while 16 primitives succeeded but required significant computation. This suggests that the optimal number of primitives is problem-dependent.
- **More primitives (32):** Provides the best balance between exploration capability and computational efficiency. The algorithm can find good trajectories quickly without excessive node exploration.
- **Computational trade-off:** There is a clear trade-off between the number of primitives and the cost per iteration. More primitives mean more trajectory evaluations, but they also enable the algorithm to find better paths with fewer total iterations.

The results demonstrate that the choice of motion primitives significantly impacts both the success rate and computational efficiency of the Kinodynamic RRT algorithm. For this particular problem, 32 primitives provides the optimal balance, achieving fast convergence with reasonable computational cost.

3.4 Path Quality Analysis

The successful paths have lengths ranging from 16.97 m to 18.45 m. The variation in path length reflects the stochastic nature of RRT and the different exploration patterns resulting from different numbers of primitives. All paths successfully navigate around obstacles while respecting the robot’s dynamic constraints, demonstrating that the algorithm produces feasible, executable trajectories.

4 Conclusion

This project successfully implemented a Kinodynamic RRT planner for a planar hover-craft robot. The implementation demonstrates that extending RRT to the kinodynamic domain enables planning of dynamically feasible trajectories that can be executed by real robots. The evaluation of different numbers of motion primitives reveals important trade-offs between exploration capability and computational efficiency, with 32 primitives providing the best overall performance for this problem.

The algorithm successfully navigates complex obstacle environments while respecting velocity and acceleration constraints, making it suitable for real-world applications such as autonomous vehicle navigation, drone path planning, and robotic manipulation tasks.